Developer-Study of DEFault++ (Version - B)

Apr 12, 2026

You are invited to take part in an online anonymous study on debugging decisions for transformer-based neural network failures.

* Required

Eligibility

Please answer the following questions to confirm whether you are eligible to participate.

1. Have you worked with transformer architectures in at least one of the following ways - training, fine-tuning, inference, debugging, or evaluation? *

- ◯ Yes
- ◯ No

2. Are you currently enrolled in a course taught by the lead researcher (Sigma Jahan), or are you directly supervised or employed by them? *

- ◯ Yes
- ◯ No

Consent Information

Please read the following information carefully before deciding whether to participate.

3. You are invited to participate in a study by Sigma Jahan, a PhD student in Computer Science at Dalhousie University, supervised by Dr. Masud Rahman. This study examines how practitioners make debugging decisions for transformer-based neural network failures. You will complete a short background questionnaire, review four debugging scenarios, choose a first inspection target and repair action, and answer short rating questions. Participation is online, anonymous, and voluntary. You may stop at any time before submitting. Since the responses are anonymous, submitted data cannot be withdrawn. Please do not include proprietary, confidential, or identifying information in open-text responses. Results will be reported only in aggregate form. Estimated time: 10 to 15 minutes.

Do you consent to participate in this study? *

- ◯ Yes
- ◯ No

Background

These questions ask about your experience with machine learning, deep learning, and transformer models.

4. Which best describes your current role? *

- [ ] Graduate student
- [ ] Research assistant
- [ ] Industry practitioner
- [ ] Other

5. How many years of machine learning or deep learning experience do you have? *

- ◯ Less than 1 year
- ◯ 1 to 2 years
- ◯ 3 to 5 years
- ◯ More than 5 years

6. Have you debugged failures in transformer models before? *

- ◯ Yes
- ◯ No

Instructions

- You will review four short debugging scenarios involving transformer model failures.
- For each scenario, you will choose a **repair action** you would try first and rate your confidence.
- Some scenarios include a diagnostic summary from **DEFault++**, an automated debugging technique for transformer faults.
- When provided, use it alongside the scenario description.
- Please note the approximate time you spent on each scenario.

7. I understand and agree *

◯ Yes

Scenario 1: Baseline + DEFault++

Cached Decoding Visibility Failure Background. When a language model generates text, it predicts one token at a time. A naive approach reprocesses the entire input from scratch at every step. A faster approach, called cached decoding, saves the internal results (key and value vectors) from previous steps and reuses them so the model only needs to process the new token.

Issue: Both approaches should produce identical output. In this model, they do not. Cached decoding gives different predictions than reprocessing from scratch.

Recent change: A shared low-level attention routine replaced an older decoding path.

Observed behavior:

- At the first generation step, the model under cached decoding acts as if it can see fewer prior tokens than it should.
- No NaN, Inf, or exceptions occur.

Logs and metrics:

- The number of prior token positions visible to the first generated token is lower than expected.
- Output difference between cached and full decoding: 0.451
- Fraction of predictions that differ: 1.000 (every single prediction changes)
-

DEFault++ output:

- Predicted fault status: Faulty
- Predicted fault category: Attention masking fault
- Predicted root cause: The causal mask has a wrong offset under cached decoding. The causal mask is what prevents each token from looking at future tokens. Under caching, the mask needs to account for the tokens already stored in the cache, and it is not doing so correctly.
- Predicted fault component: The masking logic that controls which tokens can attend to which.

8. Based on the Scenario 1: Baseline + DEFault++, what repair action would you try first? *

- ◯ **Fix the causal mask for cached decoding.** Rebuild the mask so it uses the correct positions for both the new token (query) and the stored tokens (keys) at each step. The causal mask controls which tokens the model is allowed to look at.
- ◯ **Fix the position encoding for cached decoding.** Recalculate position-based attention adjustments (such as rotary embeddings or relative position bias) using the true token positions rather than incorrect cached positions.
- ◯ **Reset the cache at every step.** Clear the saved key-value state before each new token, forcing the model to recompute from scratch. This would remove caching benefits but isolate whether the cache is the problem.
- ◯ **Adjust training hyperparameters.** Lower the learning rate and lengthen warmup before evaluating cached generation.
- ◯ Other

9. How confident are you in that choice? *

	Very confident	Confident	Moderately confident	Slightly confident	Not confident at all
Agreement	◯	◯	◯	◯	◯

10. Approximately how long did you spend on this scenario? *

	Less than a minute	1-2 minutes	3-5 minutes	5-10 minutes	More than 10 minutes
Approximate time	◯	◯	◯	◯	◯

Scenario 2 : Baseline

Position-Dependent Scoring Drift Under Cache Background. Some transformer models adjust their attention scores based on how far apart two tokens are in the sequence. For example, rotary embeddings and relative position bias both make nearby tokens attend to each other more strongly than distant ones. These adjustments are called position-dependent scoring terms.

Issue: When this model generates text by reprocessing the full input from scratch, it works correctly. When it uses cached decoding (reusing saved key-value state), the output diverges. The position-dependent scoring term becomes nearly constant under caching, as if the model has lost track of token positions.

Recent change: Cached generation support was enabled for a model that uses learned position-dependent score adjustments.

Observed behavior:

- Later tokens disagree between cached decoding and full reprocessing.
- The model is numerically stable (no NaN or Inf).
- The problem appears only when caching is active.

Logs and metrics:

- Output difference between cached and full decoding: 0.025
- Fraction of predictions that differ: 0.109
- The position-dependent score term is nearly constant under caching. It should change based on token distance but is not.

11. Based on the Scenario 2: Baseline, what repair action would you try first? *

- ◯ **Fix position distances for cached decoding.** Recalculate how far apart tokens are using their true positions in the sequence, rather than stale positions stored in the cache.
- ◯ **Fix the causal mask for cached decoding.** Rebuild the mask so that single-token decode steps can still attend to all valid prior tokens.
- ◯ **Reorder the scaling step.** Move the attention-score scaling (dividing by the square root of head dimension) to happen after the position-dependent term is added, not before.
- ◯ **Adjust training hyperparameters.** Reduce generation length and retune the learning-rate schedule.
- ◯ Other

12. How confident are you in that choice? *

	Very confident	Confident	Moderately confident	Slightly confident	Not confident at all
Agreement	◯	◯	◯	◯	◯

13. Approximately how long did you spend on this scenario? *

	Less than a minute	1-2 minutes	3-5 minutes	5-10 minutes	More than 10 minutes
Approximate time	◯	◯	◯	◯	◯

Scenario 3 : Baseline + DEFault++

Incremental State Update Mismatch Background. To generate text faster, some models save their intermediate computations (key-value vectors) and reuse them at each step instead of reprocessing the entire input. This is called incremental decoding. Each time the model predicts a new token, it should write that token's key-value vectors into the saved state so they are available for future steps.

Issue: When this model processes the entire input in one pass, it produces correct output. When it generates tokens one at a time using saved state, the predictions at the final step are different. The saved state appears to be missing some tokens.

Recent change: An incremental state reuse path was introduced to speed up generation.

Observed behavior:

- The model's raw prediction scores (logits) at the final step differ between incremental and full-pass decoding.
- No crash or exception occurs.
- The problem appears only in the incremental path.

Logs and metrics:

- Logit difference at the last step: 0.202
- Fraction of predictions that differ: 0.820
- The model appears to see fewer prior tokens than it should during incremental decoding, as if some tokens were never written into the saved state.

DEFault++ output:

- Predicted fault status: Faulty
- Predicted fault category: KV cache fault. The KV cache is where the model stores key-value vectors from previous steps so it can reuse them during generation.
- Predicted root cause: The cache update path is incorrect. The current token's key-value vectors are not being saved into the cache properly.
- Predicted fault component: The KV cache logic that saves and retrieves key-value vectors during generation.

14. Based on the Scenario 3 : Baseline + DEFault++, what repair action would you try first? *

- ◯ **Write the current token into the cache first.** Save the current token's key-value vectors into the stored state before computing attention and prediction scores, so the model can attend to all tokens including the current one.
- ◯ **Fix position encoding for cached steps.** Recalculate position-based attention adjustments using true absolute token positions instead of cached positions.
- ◯ **Tighten the causal mask.** Restrict the mask so the current token cannot attend beyond the tokens actually present in the saved state. The causal mask controls which tokens the model is allowed to look at.
- ◯ **Adjust generation settings.** Lower batch size and disable mixed precision in generation tests.
- ◯ Other

15. How confident are you in that choice? *

	Very confident	Confident	Moderately confident	Slightly confident	Not confident at all
Agreement	◯	◯	◯	◯	◯

16. Approximately how long did you spend on this scenario? *

	Less than a minute	1-2 minutes	3-5 minutes	5-10 minutes	More than 10 minutes
Approximate time	◯	◯	◯	◯	◯

Scenario 4 : Baseline

Wrong Layer Assignment for Local Attention Mode Background. Some long-context models use two kinds of attention. In *full attention,* each token can look at every prior token in the sequence. In *local attention*, each token can only look at a small window of nearby tokens (for example, the 256 closest tokens). A typical design assigns local attention to some layers and full attention to others, balancing speed and context coverage.

Issue: This model is supposed to use local attention in a specific set of layers. After a code change, local attention is activating in the wrong layers. For example, layers 0-5 might use local attention when layers 6-11 should.

Recent change: The layer-selection logic that controls which layers use local attention was refactored.

Observed behavior:

- The model runs and is numerically stable.
- Output differences appear mainly on long inputs.
- Layer-level traces show local attention is active in the wrong layers.

Logs and metrics:

- Layer trace confirms local attention activates in the wrong layer region.
- Output difference: 0.296
- Fraction of predictions that differ: 0.480

17. Based on the Scenario 4 : Baseline**,** what repair action would you try first? *

- ◯ **Fix the layer-selection condition.** Correct the logic that decides which layers use local attention so it matches the intended configuration.
- ◯ **Increase the local attention window size.** Make the window larger (allow each token to see more nearby tokens) while keeping the current layer assignment unchanged.
- ◯ **Fix position tracking in the cache for long sequences**. Rebuild how token positions are tracked in the saved state when the input is very long.
- ◯ **Adjust training hyperparameters.** Reduce the fine-tuning learning rate and extend warmup for long-context runs.
- ◯ Other

18. How confident are you in that choice? *

	Very confident	Confident	Moderately confident	Slightly confident	Not confident at all
Agreement	◯	◯	◯	◯	◯

19. Approximately how long did you spend on this scenario? *

	Less than a minute	1-2 minutes	3-5 minutes	5-10 minutes	More than 10 minutes
Approximate time	◯	◯	◯	◯	◯

DEFault++ Output Evaluation

20. Indicate your agreement with each statement *

	Strongly agree	Agree	Neither agree nor disagree	Disagree	Strongly disagree
The DEFault++ output was easy to understand.	◯	◯	◯	◯	◯
The DEFault++ output helped me decide what repair to try first.	◯	◯	◯	◯	◯
I would use a tool like this in practice.	◯	◯	◯	◯	◯

21. Which condition better supported your debugging decision? *

- ◯ Baseline
- ◯ DEFault++ assisted
- ◯ About the same

22. Which condition felt faster? *

- ◯ Baseline
- ◯ DEFault++ assisted
- ◯ About the same